Section 11.8 of 10: Context Engineering — Dynamic Token Budget Management

Executive Overview

Context engineering is the discipline of deciding *what goes into the model's context window* at each turn. With models supporting 128K–1M token windows, the naive approach of "put everything in" fails on cost, latency, and attention quality. This section covers budget allocation, compression strategies, position-aware context assembly, and cache-aware prompting — all essential for production AI systems.

> ***Math-heavy alert:*** *Token budget arithmetic is frequently tested. Worked examples show exactly how to allocate 128K tokens across competing context types and calculate cost savings from caching.*

11.8.1 Context Window Budget Allocation

Dynamic Budget Model

```
class ContextBudget:
    TOTAL = 128_000  # Claude Sonnet 4, GPT-4o

    # Fixed reservations (always present)
    RESERVED = {
        "system_prompt":     2_500,   # Static, cache-eligible
        "current_user_turn": 1_000,   # Current query
        "output_reserve":    4_000,   # Space for model response
    }

    # Dynamic allocation by task type
    TASK_BUDGETS = {
        "coding": {
            "rag_context":    50_000,  # Large codebases need full file context
            "examples":       20_000,
            "history_recent": 10_000,
            "scratchpad":     40_500,  # Remaining
        },
        "chat": {
```

```
            "rag_context":    30_000,
            "examples":       10_000,
            "history_recent": 20_000,
            "scratchpad":     60_500,
        },
        "analysis": {
            "rag_context":    70_000,  # Full document analysis
            "examples":        5_000,
            "history_recent":  8_000,
            "scratchpad":     39_500,
        }
    }
```

Worked numeric example for 128K window:

```
System prompt:          2,500 tokens  (cached → ~0 cost after first use)
Recent history:        10,000 tokens
RAG context (coding):  50,000 tokens
Code examples:         20,000 tokens
Current query:          1,000 tokens
Output reserve:         4,000 tokens
──────────────────────────────────────
Total:                 87,500 tokens  (68% utilized, 12.5K headroom)
```

11.8.2 Position-Based Attention Management

Lost-in-the-Middle Problem

Research shows model attention is highest at the start and end of context, with a measurable dip in the middle (the "lost-in-the-middle" effect). For 32K+ contexts, critical information buried in the middle may be underweighted.

Evidence: In a 20-document RAG setup, accuracy drops from 71% (document at position 1) to 45% (document at position 10) to 68% (document at position 20).

Optimal Context Ordering

```
Position 1 (highest attention):
  System prompt + task framing
  → Always first; always cached

Position 2 (high attention):
  Most relevant retrieved chunks (top-3 by score)
  Current query restatement

Middle section (attention dip — minimize critical info here):
```

```
  Supporting context (moderate relevance chunks)
  Historical conversation (summarized)

Final positions (high attention):
  Current user message (most recent)
  Explicit instruction: "Based on the above, answer:"
```

Recency-Weighted Assembly Algorithm

```
def assemble_context(query, candidates, budget):
    scored = []
    for chunk in candidates:
        score = (
            embedding_similarity(query, chunk)  * 0.50 +  # Semantic relevance
            recency_weight(chunk.timestamp)     * 0.20 +  # Recent = higher weight
            source_quality(chunk.source)        * 0.20 +  # Trusted sources
            user_feedback(chunk.id)             * 0.10    # Past positive signals
        )
        scored.append((chunk, score))

    scored.sort(key=lambda x: x[1], reverse=True)

    # Fill budget greedily, highest score first
    selected, tokens_used = [], 0
    for chunk, score in scored:
        if tokens_used + chunk.token_count <= budget:
            selected.append(chunk)
            tokens_used += chunk.token_count

    # Apply position optimization:
    # Top-3 chunks → start of context
    # Remaining chunks → middle
    # Always end with current query
    return position_optimize(selected)
```

11.8.3 Compression Strategies

Hierarchical Summarization

```
Turn summarization hierarchy:

Level 1: Individual turn → 100-token summary
  Original turn (500 tokens): "User asked about cancellation policy.
    Agent provided 3 options: full refund within 30 days, 50% refund
    31-60 days, no refund after 60 days. User asked about exceptions
    for medical emergencies. Agent escalated to check policy..."
  Summary: "Discussed cancellation policy; exception requested for
```

```
    medical reasons; pending policy check."

Level 2: 5 summaries → 250-token meta-summary
Level 3: 10 meta-summaries → 500-token archive summary

Trigger: When context exceeds 80% of budget
Action: Summarize oldest N turns first (LRU-style)
```

Compression Method Comparison

Method	Compression	Accuracy Loss	Latency Overhead	Use When
None	1×	0%	0ms	Context fits in budget
Extractive	3×	5–10%	+50ms	Fast, moderate compression
Recursive summary	5×	10–20%	+200ms	Long conversations
Abstractive	10×	30–50%	+500ms	Archive / very old history
Entity extraction	20×+	Depends on task	+100ms	Structured knowledge retention

Proof by contradiction for over-compression: Suppose you use 10× abstractive compression for all history. A 50-turn conversation about debugging a specific error gets compressed to ~5 sentences. When the same error recurs in turn 60, the agent lacks the specific traceback and attempted fixes — it re-diagnoses from scratch, wasting 10 turns. Entity extraction preserving `{"error": "OOM", "attempted_fixes": ["increase heap", "reduce batch size"]}` is far more useful than a prose summary.

11.8.4 Cache-Aware Prompting

Prompt Caching Mechanics

Most LLM providers (Anthropic, OpenAI) cache the *prefix* of prompts when the same prefix is repeated across requests.

```
Cached (stable, written once):
──────────────────────────────────────────────────
You are a customer support agent for Acme Corp.
[2,500 tokens of product knowledge, policies, tone guidelines]
──────────────────────────────────────────────────

Not cached (dynamic per request):
──────────────────────────────────────────────────
Customer: "How do I cancel my subscription?"
Previous conversation: [summary of last 3 turns]
Retrieved context: [relevant FAQ chunks]
──────────────────────────────────────────────────
```

Cache Hit Rate Calculation

```
Scenario: 10,000 requests/day
System prompt: 2,500 tokens (always identical)
Dynamic context: 5,000 tokens average

Without caching:
  Input cost = 10,000 × 7,500 tokens × $3.00/M = $225/day

With 95% cache hit on system prompt:
  Cached tokens: 2,500 × 0.95 × 10,000 = 23.75M tokens @ $0.30/M = $7.13
  Uncached tokens: 5,000 × 10,000 tokens = 50M @ $3.00/M = $150
  Uncached prefix miss: 2,500 × 500 × $3.00/M = $3.75
  Total: $160.88/day

Savings: $225 - $161 = $64/day = ~28% cost reduction
```

Maximizing cache hit rate: 1. Keep system prompt identical across all requests (no dynamic content in system prompt) 2. Place frequently-repeated content earliest in the prompt 3. Avoid timestamps or request IDs in cacheable prefix 4. Use consistent formatting (even whitespace changes break cache)

Interview Problems

Problem 1: Long-Running Research Agent Context Management

Scenario: A research agent runs for 2 hours, making 200+ tool calls. By turn 50, context is full. Design a system to prevent overflow while retaining relevant information.

Solution:

```
class ResearchContextManager:
    BUDGET = 100_000  # Reserve 28K for system + output

    def __init__(self):
        self.working_memory = []     # Last 10 turns, full detail
        self.episodic_cache = {}     # Redis: compressed summaries
        self.semantic_store = VectorStore()  # Key facts as embeddings

    def assemble_context(self, query: str) -> str:
        # 1. Always include: system + current query
        context_parts = [system_prompt, query]  # ~3,500 tokens

        # 2. Retrieve semantically relevant facts
        relevant_facts = self.semantic_store.search(query, k=20)
        context_parts.append(format_facts(relevant_facts))  # ~10,000 tokens

        # 3. Recent turns (last 10, full detail)
        context_parts.append(format_turns(self.working_memory[-10:]))  # ~15,000
tokens

        # 4. Recent summaries (compressed older turns)
        summaries = self.episodic_cache.get_recent(n=5)
        context_parts.append(format_summaries(summaries))  # ~5,000 tokens

        total = sum(count_tokens(p) for p in context_parts)
        assert total < self.BUDGET, f"Budget exceeded: {total}"

        return "\n\n".join(context_parts)

    def on_turn_complete(self, turn: dict):
        self.working_memory.append(turn)

        # Compress when working memory exceeds 10 turns
        if len(self.working_memory) > 10:
            old_turns = self.working_memory[:5]

            # Extract facts before summarizing
            facts = extract_facts(old_turns)
# e.g., {"finding": "...", "source": "..."}
            for fact in facts:
                self.semantic_store.upsert(fact)

            # Compress and move to episodic
            summary = summarize_turns(old_turns)
            self.episodic_cache.set(f"summary_{turn['id']}", summary)

            # Keep only recent turns in working memory
            self.working_memory = self.working_memory[5:]
```

Problem 2: Reducing Context Costs by 50% for a High-Volume Chatbot

Scenario: A chatbot sends 5M requests/day with 8K average input tokens. Monthly bill is $36,000. Reduce cost by 50% without sacrificing quality.

Solution:

```
Current cost:
  5M requests × 8,000 tokens × $3.00/M tokens = $120,000/month

Strategy 1: Prompt caching (system prompt = 2,000 tokens)
  Cache hit rate target: 90%
  Savings: 2,000 tokens × 0.9 × 5M × ($3.00 - $0.30)/M = $24,300/month

Strategy 2: Semantic caching (cache identical queries)
  ~30% of queries are duplicates or near-duplicates
  Savings: 5M × 0.30 × 8,000 × $3.00/M = $36,000/month
  Cost of cache infrastructure: ~$500/month

Strategy 3: Context compression for long conversations
  Only 20% of sessions have >5 turns (when history becomes large)
  Compress history in long sessions: 8K → 4K average for those sessions
  Savings: 5M × 0.20 × 4,000 tokens × $3.00/M = $12,000/month

Combined savings:
  $24,300 + $36,000 + $12,000 = $72,300/month (60% reduction!)

Implementation priority:
  1. Semantic cache (highest ROI, easiest to implement)
  2. Prompt caching (zero code change if prompt already stable)
  3. Context compression (highest complexity, implement last)
```

Problem 3: Handling Context Clashes (Contradictory Retrieved Information)

Scenario: A RAG system retrieves two chunks that contradict each other (e.g., policy updated last month vs. old policy). The model silently picks one. How do you surface and resolve this?

Solution:

```
def resolve_context_clashes(chunks: List[Chunk], query: str) -> List[Chunk]:
    # Step 1: Detect contradictions using embedding clustering
    embeddings = [embed(c.text) for c in chunks]
    contradictions = find_contradictory_pairs(chunks, embeddings)

    for pair in contradictions:
        older, newer = sorted(pair, key=lambda c: c.timestamp)

        # Step 2: Annotate — don't silently drop either
```

```
            newer.metadata["supersedes"] = older.id
            older.metadata["superseded_by"] = newer.id
            older.text = f"[OUTDATED as of {newer.timestamp}] {older.text}"

    # Step 3: Sort: newer sources first, annotated older ones after
    chunks_sorted = sorted(chunks, key=lambda c: c.timestamp, reverse=True)

    # Step 4: Add explicit contradiction notice to prompt
    if contradictions:
        notice = Chunk(
            text=f"NOTE: {len(contradictions)} contradictions found in sources. "
                 f"Prefer the most recent source dated
{chunks_sorted[0].timestamp}.",
            token_count=50,
            source="system"
        )
        return [notice] + chunks_sorted

    return chunks_sorted
```

Why not just drop the older chunk: Silent removal hides the fact that information changed. If the user is aware of the old policy, they may distrust an answer that ignores it. Making the contradiction explicit lets the model reason about the change rather than blindly using one source.